



Python

# Strings

Operators & Functions

```
# Creating strings
```

```
s = "Hello!"
```

```
name = 'Python'
```

```
# Concatenation
```

```
msg = s + " " + name
```

```
# Length
```

```
print(len(msg))
```



# What is a String?

## Definition

A string is a sequence of characters enclosed in single, double, or triple quotes.

## Immutable

Strings in Python are immutable — once created, they cannot be changed in place.

## Data Type

Strings belong to the str type and can hold letters, digits, spaces, and symbols.

## ► Creating Strings

```
s1 = 'Hello'
s2 = "World"
s3 = '''Multi
line'''
```

```
# Check type
```

```
print(type(s1))
# <class 'str'>
```

```
# Indexing
```

```
print(s1[0])    # H
print(s1[-1])   # o
```

# What is a String?

## Ordered

characters have a defined position (index)

## Iterable

can be looped over character by character

## ► Creating Strings

```
s1 = 'Hello'  
s2 = "World"  
s3 = '''Multi  
line'''
```

```
# Check type
```

```
print(type(s1))  
# <class 'str'>
```

```
# Indexing
```

```
print(s1[0])    # H  
print(s1[-1])   # o
```

- Individual characters of a String can be accessed by using the method of **Indexing**. Indexing allows negative address references to access characters from the back of the String
- e.g. -1 refers to the last character, -2 refers to the second last character and so on
- While accessing an index out of the range will cause an **IndexError**. Only Integers are allowed to be passed as an index, float or other types will cause a **TypeError**.

|     |     |     |     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| 0   | 1   | 2   | 3   | 4   | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| P   | y   | t   | h   | o   | n  |    | p  | r  | o  | g  | r  | a  | m  |
| -14 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

# Python Program to Access  
# characters of String

```
String1 = "Python Program"  
print("Initial String: ")  
print(String1)
```

# Printing First character

```
print("\nFirst character of String is: ")  
print(String1[0])
```

# Printing Last character

```
print("\nLast character of String is: ")  
print(String1[-1])
```

The following operators can be applied to sequence types, including strings, lists, and tuples:



| Operation                              | Description                                       |
|----------------------------------------|---------------------------------------------------|
| $s + r$                                | Concatenation                                     |
| $s * n, n * s$                         | Makes $n$ copies of $s$ , where $n$ is an integer |
| $v1, v2..., vn = s$                    | Variable unpacking                                |
| $s[i]$                                 | Indexing                                          |
| $s[i:j]$                               | Slicing                                           |
| $s[i:j:stride]$                        | Extended slicing                                  |
| $x \text{ in } s, x \text{ not in } s$ | Membership                                        |
| for $x$ in $s$ :                       | Iteration                                         |
| $\text{all}(s)$                        | Returns True if all items in $s$ are true.        |
| $\text{any}(s)$                        | Returns True if any item in $s$ is true.          |
| $\text{len}(s)$                        | Length                                            |
| $\text{min}(s)$                        | Minimum item in $s$                               |
| $\text{max}(s)$                        | Maximum item in $s$                               |
| $\text{sum}(s [, initial])$            | Sum of items with an optional initial value       |

```
name = "Robert" + "Alice"  
print(name)
```

different ways to concatenate strings:

Using + operator

Using join() method

Using % operator

Using format() function

Using f'string

```
name = "Alice"
```

```
msg = f"Hi {name}! How  
are you"
```

```
print(msg)
```

# String Operators



## **+** Concatenation

```
"Hi" + " " + "there"
```

→ "Hi there"

## **\*** Repetition

```
"ha" * 3
```

→ "hahaha"

## **in** Membership

```
"py" in "python"
```

→ True

## **not in** Not Membership

```
"z" not in "abc"
```

→ True

## **[]** Indexing

```
"hello"[1]
```

→ "e"

## **[:]** Slicing

```
"hello"[1:4]
```

→ "ell"

# String Slicing `s[start : end : step]`



P Y T H O N

|            |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|
| (+) index: | 0  | 1  | 2  | 3  | 4  | 5  |
| (-) index: | -6 | -5 | -4 | -3 | -2 | -1 |

`s[0:3]`

→ "PYT"

First 3 chars

`s[2:5]`

→ "THO"

Chars at index 2-4

`s[:4]`

→ "PYTH"

Start to index 3

`s[3:]`

→ "HON"

Index 3 to end

`s[-3:]`

→ "HON"

Last 3 chars

`s[::-2]`

→ "PTO"

Every 2nd char

`s[::-1]`

→ "NOHTYP"

Reverse string

`s[1:5:2]`

→ "YH"

Index 1-4, step 2

# String Slicing (:)

- We can return a range of characters by using the slice operator(:)
- Example `s=str[start:end]`
- Get the characters from **start to end-1**
- Specify the start index and the end index, separated by a colon, to return a part of the string

```
str = 'Hello Python'  
s= str[1:4]  
'ell'
```

## Examples

```
str[:5]      #Hello  
str[2:]      # llo Python
```

# Stride in a String Slice

- Adding an additional **:** and a third index designates a **stride** (also called a step), which indicates how many characters to jump after retrieving each character in the slice
- **s[i:j:stride]**

```
str = 'Hello Python'  
s= str[0:11:2]
```

# Built-in String Methods (Part 1)



| Method              | Syntax                         | Example                               | Output                       |
|---------------------|--------------------------------|---------------------------------------|------------------------------|
| <b>upper()</b>      | <code>str.upper()</code>       | <code>"hello".upper()</code>          | <code>"HELLO"</code>         |
| <b>lower()</b>      | <code>str.lower()</code>       | <code>"WORLD".lower()</code>          | <code>"world"</code>         |
| <b>strip()</b>      | <code>str.strip()</code>       | <code>" hi ".strip()</code>           | <code>"hi"</code>            |
| <b>split()</b>      | <code>str.split(sep)</code>    | <code>"a,b,c".split(",")</code>       | <code>['a', 'b', 'c']</code> |
| <b>replace()</b>    | <code>str.replace(o,n)</code>  | <code>"cat".replace("c", "b")</code>  | <code>"bat"</code>           |
| <b>find()</b>       | <code>str.find(sub)</code>     | <code>"python".find("th")</code>      | <code>2</code>               |
| <b>count()</b>      | <code>str.count(sub)</code>    | <code>"banana".count("a")</code>      | <code>3</code>               |
| <b>startswith()</b> | <code>str.startswith(s)</code> | <code>"hello".startswith("he")</code> | <code>True</code>            |

# Built-in String Methods (Part 2)



| Method            | Syntax                        | Example                                | Output                     |
|-------------------|-------------------------------|----------------------------------------|----------------------------|
| <b>endswith()</b> | <code>str.endswith(s)</code>  | <code>"hello".endswith("lo")</code>    | <code>True</code>          |
| <b>join()</b>     | <code>sep.join(list)</code>   | <code>"-".join(["a", "b", "c"])</code> | <code>"a-b-c"</code>       |
| <b>isdigit()</b>  | <code>str.isdigit()</code>    | <code>"123".isdigit()</code>           | <code>True</code>          |
| <b>isalpha()</b>  | <code>str.isalpha()</code>    | <code>"abc".isalpha()</code>           | <code>True</code>          |
| <b>title()</b>    | <code>str.title()</code>      | <code>"hello world".title()</code>     | <code>"Hello World"</code> |
| <b>center()</b>   | <code>str.center(w, c)</code> | <code>"hi".center(8, "*")</code>       | <code>****hi****</code>    |
| <b>zfill()</b>    | <code>str.zfill(w)</code>     | <code>"42".zfill(5)</code>             | <code>"00042"</code>       |
| <b>format()</b>   | <code>str.format()</code>     | <code>"{} {}".format("Hi", "!")</code> | <code>"Hi !"</code>        |

# Built-in string methods



|                                                                    |                                                                                                                         |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <u><a href="#">capitalize()</a></u>                                | Converts the first character to upper case                                                                              |
| <u><a href="#">casefold()</a></u> / <u><a href="#">lower()</a></u> | Converts string into lower case                                                                                         |
| <u><a href="#">find()</a></u>                                      | Searches the string for a specified value and returns the position of where it was found. Otherwise -1                  |
| <u><a href="#">index()</a></u>                                     | Searches the string for a specified value and returns the position of where it was found. Otherwise raises an exception |
| <u><a href="#">count()</a></u>                                     | Returns the number of times a specified value occurs in a string                                                        |
| <u><a href="#">center()</a></u>                                    | Returns a centered string                                                                                               |
| <u><a href="#">endswith()</a></u>                                  | Returns true if the string ends with the specified value                                                                |
| <u><a href="#">replace()</a></u>                                   | Returns a string where a specified value is replaced with a specified value                                             |
| <u><a href="#">upper()</a></u>                                     | Converts a string into upper case                                                                                       |

# Built-in string methods



|                           |                                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------|
| <code>join()</code>       | Converts the elements of an iterable into a string                                            |
| <code>split()</code>      | Splits the string at the specified separator, and returns a list                              |
| <code>startswith()</code> | Returns true if the string starts with the specified value                                    |
| <code>swapcase()</code>   | Swaps cases, lower case becomes upper case and vice versa                                     |
| <code>title()</code>      | Converts the first character of each word to upper case                                       |
| <code>strip()</code>      | Returns a trimmed version of the string                                                       |
| <code>rfind()</code>      | Searches the string for a specified value and returns the last position of where it was found |
| <code>rindex()</code>     | Searches the string for a specified value and returns the last position of where it was found |

# Built-in string methods



|                    |                                                                  |
|--------------------|------------------------------------------------------------------|
| <b>isalnum()</b>   | Returns True if all characters in the string are alphanumeric    |
| <b>isalpha()</b>   | Returns True if all characters in the string are in the alphabet |
| <b>isdecimal()</b> | Returns True if all characters in the string are decimals        |
| <b>islower()</b>   | Returns True if all characters in the string are lower case      |
| <b>isupper()</b>   | Returns True if all characters in the string are upper case      |

# String methods



## capitalize() Method

Upper case the first letter in this sentence:

```
string.capitalize()
```

### Syntax:

```
txt = "welcome to python programming."  
x = txt.capitalize()  
print (x)
```

The upper() method returns a string where all characters are in upper case.

```
string.upper()
```

### Syntax:

# String methods



## replace() method

The replace() method replaces a specified phrase with another specified phrase.

```
string.replace(oldvalue, newvalue, count)
```

### Syntax:

| Parameter       | Description                                                                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------------------|
| <i>oldvalue</i> | Required. The string to search for                                                                                  |
| <i>newvalue</i> | Required. The string to replace the old value with                                                                  |
| <i>count</i>    | Optional. A number specifying how many occurrences of the old value you want to replace. Default is all occurrences |

```
txt = "I like Python"  
x = txt.replace("Python", "Programming")
```

# String methods



## find() Method

The find() method return the index of first occurrence of the specified value. The find() method returns -1 if the value is not found.

Syntax:

**string.find(value, start, end)**

```
str = 'i love python program'
```

```
# returns first occurrence of Substring
```

```
index = str.find('python')
```

```
print ("Substring 'python' found at index:", index )
```

Optional. Where to start

Optional. Where to end

# String methods



## index() Method

The index() method finds the first occurrence of the specified value. The index() method raises an exception if the value is not found..

Syntax:

`string.index(value, start, end)`

```
str = 'i love python program'
```

```
# returns first occurrence of Substring  
index = str.find('python')  
print ("Substring 'python' found at index:", index )
```

Optional. Where to start

Optional. Where to end

# String methods



## join() Method

The join() method is a string method and returns a string in which the elements of sequence have been joined by str separator

Syntax:

```
string.join(iterable)
```

```
names = ("John", "Peter", "Vicky")  
x = "#".join(names)  
print(x)
```

# String methods



## split() Method

The split() method splits a string into a list.

You can specify the separator, default separator is any whitespace.

Syntax:

```
string.split(separator, maxsplit)
```

**separator** Optional. Specifies the separator to use when splitting the string. By default any whitespace is a separator

**maxsplit** Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences"

```
txt = "apple#banana#cherry#orange"  
x = txt.split("#")  
print(x)
```

OUTPUT

```
['apple', 'banana', 'cherry', 'orange']
```

```
txt = "apple#banana#cherry#orange"  
x = txt.split("#",1)  
print(x)
```

OUTPUT

```
['apple', 'banana#cherry#orange']
```

# String methods



## count() Method

The count() method returns the number of times a specified value appears in the string.

Syntax:

```
string.count(value, start, end)
```

```
str = 'i love python, python is a programing language'  
x = str.count("python")  
print(x)
```

# String methods



## center() Method

The center() method will center align the string

Syntax:

```
string.center(length, character)
```

length - Required. The length of the returned string

character - Optional. The character to fill the missing space on each side. Default is " " (space)

```
txt = "python"  
x = txt.center(20)  
print(x)
```

Print the word "python", taking up the space of 20 characters, with "python" in the middle

# Summary



Strings are sequences of characters — immutable and ordered

Use + for concatenation and \* for repetition

Slicing `s[start:end:step]` gives powerful substring control

Methods like `split()`, `join()`, `replace()`, `find()` transform strings

f-Strings are the modern way to embed variables in strings

Use `in` / `not in` to test substring membership